

Relatório Técnico — TP1: From Raw Detections to Real Intelligence

Unidade Curricular: Interação com Modelos de Larga Escala

Trabalho realizado por: Simão Abranches nº53682

Ano Letivo: 2025/2026

1. Introdução

1.1 Descrição do Problema

Este trabalho aborda o problema de transformar dados brutos de visão computacional numa loja de retalho em inteligência operacional utilizável por um gestor. O dataset cobre 7 dias consecutivos de operação (10 a 16 de março de 2025), com aproximadamente 250.000 eventos gerados por um sistema de deteção em 23 zonas da loja.

O desafio fundamental é que os dados chegam sem identidade: o sistema sabe que alguém entrou na zona Z_S3 às 14h32, mas não sabe se é a mesma pessoa que esteve em Z_N2 às 14h28. O percurso individual tem de ser reconstruído a partir de um stream de eventos anónimos e intercalados.

1.2 Estrutura do repositório

```
tp1-liacd/
├── data/
│   └── events.csv          # Dataset original de entrada
├── src/
│   ├── stitcher.py        # Fase 1: Reconstrução de trajetórias
│   ├── analytics.py       # Fase 2.1: Cálculo de métricas e anomalias
│   ├── insights.py        # Fase 2.2: Motor de insights (LLM)
│   └── report.py          # Fase 2.3: Gerador do relatório Markdown
├── prompts/
│   ├── prompt_A_zero_shot.txt # Estratégia de prompting inicial
│   └── prompt_B_few_shot.txt  # Estratégia otimizada com exemplos
├── output/
│   ├── journeys.csv       # Output do stitching
│   ├── metrics.json       # Output do analytics (Source of Truth)
│   ├── insights.json      # Output da LLM em formato estruturado
│   └── weekly_report.md    # Relatório final para o gestor
├── requirements.txt        # Dependências do projeto (pip)
├── README.md              # Instruções de instalação e execução
└── evaluate.py            # Harness de avaliação oficial
```

1.3 Dificuldades Identificadas no Dataset

Após análise exploratória do **events.csv**, identificaram-se as seguintes dificuldades:

Ausência de identidade. Não existe **person_id**. Múltiplas pessoas estão presentes simultaneamente na loja e os seus eventos estão intercalados por ordem cronológica, tornando a associação não trivial.

Erros de classificação demográfica. O classificador de género apresenta uma taxa de erro de aproximadamente 8% e o de faixa etária de 12%. Isto significa que dois eventos da mesma pessoa podem ter atributos diferentes, complicando a associação por atributos.

Densidade temporal elevada. Com 250.015 eventos em 84 horas de operação (12h/dia × 7 dias), a média é de aproximadamente 2.976 eventos por hora, o que implica muitas pessoas em simultâneo em cada zona e colisões frequentes de atributos.

Eventos sem estrutura garantida. A sequência **entry** → **linger** → **exit** é esperada mas não garantida. Sensores podem falhar, pessoas podem sair por zonas não instrumentadas, e o evento **linger** pode aparecer sem o **exit** correspondente.

1.4 Decisão de Arquitetura: Separação Python/LLM

A decisão arquitetural mais importante deste trabalho foi a LLM nunca acede diretamente ao CSV nem executa cálculos matemáticos. Toda a computação sobre os dados é feita em Python, e a LLM recebe apenas métricas pré-calculadas em formato JSON.

Esta separação justifica-se por três razões:

1. **Fiabilidade numérica.** LLMs são notoriamente imprecisas em aritmética. Delegar cálculos à LLM introduziria erros sistemáticos nos factos apresentados ao gestor.
2. **Verificabilidade.** Com métricas calculadas em Python, cada número no relatório final é rastreável ao código que o produziu. É possível auditar o pipeline e detetar alucinações.
3. **Eficiência.** Modelos locais com 8B parâmetros têm contexto windows limitados. Enviar 250.000 linhas de CSV seria impossível; enviar um JSON de métricas é viável.

2. Algoritmo de Stitching

2.1 Descrição do Algoritmo

O algoritmo implementado em **stitcher.py** é uma abordagem greedy de associação temporal, ou seja, os eventos são processados em ordem cronológica e, para cada novo evento entry, o algoritmo tenta associá-lo à trajetória aberta mais compatível. Se não encontrar nenhuma compatível, cria uma nova trajetória.

O algoritmo mantém duas listas: **open_trajectories** (pessoas ainda dentro da loja) e **closed_trajectories** (pessoas que já saíram). Uma trajetória é fechada quando:

- É detetado um **exit** numa zona **Z_E** ou **Z_CK**
- O tempo desde o último evento excede **MAX_GAP_SECONDS** (120 segundos)

2.2 Heurísticas Implementadas

Consistência temporal — foi implementada como invariante do algoritmo, ou seja, um entry só pode ser associado a uma trajetória que não tenha uma visita atualmente aberta (sem exit correspondente). Isto garante que nenhuma pessoa é colocada em duas zonas ao mesmo tempo.

Plausibilidade de transição — o tempo entre o exit de uma zona e o entry da zona seguinte deve ser inferior a 120 segundos, optei por este tempo por achar mais correspondente com a realidade. Trajetórias sem atividade durante este período são automaticamente fechadas e o evento inicia uma nova trajetória.

Consistência de atributos — o género é tratado como sinal forte: se o género do evento é incompatível com o da trajetória, o score retorna -1.0 e a associação é rejeitada. A faixa etária é tratada com mais tolerância, aceitando categorias adjacentes (e.g., adult e middle_aged) com penalidade reduzida, dado o maior erro do classificador (12%).

Plausibilidade de sequência — transições entre tipos de zona que seguem o percurso natural de compras ($Z_E \rightarrow Z_N \rightarrow Z_S \rightarrow Z_C \rightarrow Z_{CK}$) recebem um bônus de 0.1 no score. Transições regressivas (e.g., $Z_{CK} \rightarrow Z_S$) não recebem bônus.

2.3 Função de Score

O score de compatibilidade entre uma trajetória e um evento combina três factores:

$$\text{score} = (\text{time_score} \times 0.55) + (\text{gender_score} \times 0.25) + (\text{age_score} \times 0.10) + \text{sequence_bonus}$$

Onde $\text{time_score} = 1 - \text{gap} / \text{MAX_GAP_SECONDS}$, normalizado entre 0 e 1. O tempo tem o maior peso (55%) porque é o sinal mais fiável — ao contrário dos atributos demográficos, o tempo não está sujeito a erros de classificação.

2.4 Métricas de Qualidade Obtidas

Métrica	Valor	Interpretação
Consistência	100%	Nenhuma pessoa foi colocada em duas zonas em simultâneo
Cobertura	≈ 100%	85.347 visitas reconstruídas de 85.347 eventos entry
Compleitude	13.8%	3.911 de 28.394 trajectórias com Z_E no início e Z_E/Z_CK no fim
Visitantes únicos	28.370	Ao longo de 7 dias
Gap mediano	7s	Plausível para transição entre zonas adjacentes
Gap p90	23s	90% das transições ocorrem em menos de 23 segundos

2.5 Análise da Compleitude

Apesar de ter conseguido consistência e cobertura de 100% , a completude de 13,8% foi o valor mais preocupante, uma trajectória é considerada completa se começa numa zona Z_E e termina numa zona Z_E ou Z_CK. A baixa completude foi devido às seguintes causas:

1. **Sensores de entrada com falhas.** Muitas trajectórias começam numa zona interior (Z_N ou Z_S) porque o evento de entry na porta não foi detetado. O stitcher cria uma trajectória na mesma para não perder cobertura, mas essas trajectórias não são "completas" por definição.
2. **Saídas não detetadas.** Pessoas que saem por saídas secundárias ou em zonas sem sensor de saída ficam com trajectória aberta, que é forçosamente fechada pelo timeout de 120 segundos.

Esta limitação é honesta e esperada num sistema de visão computacional no mundo real.

2.6 Casos de Falha

O algoritmo pode errar em dois cenários:

- **Fusão incorrecta:** duas pessoas com o mesmo género e faixa etária que passam pela mesma zona em sequência com menos de 120 segundos de diferença podem ser erroneamente fundidas numa trajectória.
- **Fragmentação excessiva:** uma pessoa que demora mais de 2 minutos entre duas zonas (e.g., parou no corredor sem sensor) terá a trajectória quebrada em dois fragmentos.

3. Pipeline Analítico

3.1 Métricas Calculadas

O **analytics.py** calcula cinco famílias de métricas a partir do **journeys.csv**:

Tráfego geral: visitantes únicos por dia e por hora; hora de pico; duração média e mediana de visita; dia mais e menos movimentado.

Métricas por zona: tráfego total e visitantes únicos por zona, dwell time médio (calculado apenas sobre eventos com $dwell_s > 0$, i.e., com linger associado), taxa de paragem (visitantes com linger / visitantes totais); bi-gramas e tri-gramas de sequências de zonas mais frequentes.

Funil de cliente: da entrada (Z_E) à caixa (Z_C/Z_CK), com contagem em cada tipo de zona, taxa de conversão, perfil demográfico de quem não chegou à caixa, última zona antes de sair para quem não comprou.

Segmentação demográfica: distribuição de género e faixa etária global, distribuição por hora do dia, dwell time médio por segmento e por zona.

Deteção de anomalias: para cada zona e cada hora, é calculada a média e o desvio-padrão dos primeiros 6 dias, identifica-se onde o dia 7 se desvia mais de 2σ da média.

3.2 Decisões de Design

Dwell apenas com linger. Linhas com $dwell_s = 0$ são entradas/saídas sem permanência real. Incluí-las na média de dwell enviesaria significativamente o resultado. Apenas eventos com $dwell_s > 0$ contribuem para a média de dwell por zona.

Visitante único por dia. Um **person_id** que entra e sai duas vezes no mesmo dia conta como 1 visitante único para esse dia. Isto reflete a realidade do negócio — o gestor quer saber quantas pessoas distintas visitaram a loja.

Dia N-1 como target de anomalias. Em vez de fixar "dia 7 = domingo", o algoritmo usa sempre o último dia do dataset como target e os restantes como baseline. Isto garante que o harness de avaliação funciona com qualquer dataset sem alterações ao código.

3.3 Formato do metrics.json

O ficheiro está estruturado em cinco secções de topo (traffic, zones, funnel, demographics, anomalies) mais uma secção dataset_info com metadados. A LLM recebe uma versão filtrada deste JSON — apenas os factos mais relevantes — através da função `extract_key_facts()`, que selecciona top 5 zonas, top 5 anomalias por z-score, funil resumido e distribuição demográfica global.

4. Engenharia de Prompts

4.1 Estratégia A — Zero-shot

A estratégia A envia à LLM o schema de output, os dados resumidos e uma instrução directa:

És um analista de retalho experiente. Recebes métricas calculadas automaticamente de uma loja durante uma semana e deves gerar insights acionáveis em português. Usa apenas os números que estão nos dados fornecidos. Responde exclusivamente em json válido.

Resultado obtido: 0 insights, erro de parse. O modelo llama3.1:8b não conseguiu gerar JSON válido de forma consistente sem exemplos, produzindo texto com explicações em vez de JSON puro. Tempo de execução: 155.8 segundos.

4.2 Estratégia B — Few-shot

A estratégia B adiciona, antes do pedido, três pares de exemplos de maus vs. bons insights, retirados diretamente do enunciado e expandidos. Os exemplos ilustram explicitamente o que distingue um insight genérico de um acionável:

Mau insight	Bom insight
"A zona teve bastante tráfego."	"Z_S3 teve 847 visitantes na quinta, 31% acima da média semanal."
"Recomenda-se melhorar o atendimento."	"Às quartas 12h-13h, tempo médio de espera em Z_C2 foi 4,2 min."
"Houve uma anomalia."	"No domingo às 16h, Z_N4 teve 0 visitantes contra média de 23 (4.0σ)."

Resultado obtido: 4 insights válidos, JSON corretamente estruturado. Tempo de execução: 98.6 segundos.

4.3 Comparação Quantitativa A vs. B

Métrica	Estratégia A	Estratégia B
Insights gerados	0	4
Erro de parse	Sim	Não
Tempo de execução	155.8s	98.6s
Observações com números	N/A	4/4 (100%)
Cobertura de categorias	N/A	tráfego, zona, funil, anomalia

A estratégia B foi claramente superior em todos os aspectos. O facto de a estratégia A ter falhado completamente, foi uma limitação do sistema: modelos locais de 8B parâmetros têm capacidade limitada para seguir instruções complexas de formatação sem exemplos concretos.

4.4 Iterações Realizadas

A primeira versão dos prompts enviava o **metrics.json** completo à LLM. Isso resultava em timeouts e respostas truncadas devido ao context window limitado do llama3.1:8b. A solução foi criar a função `extract_key_facts()` que filtra apenas os dados mais relevantes antes de enviar ao modelo, reduzindo o input em aproximadamente 80%.

5. Avaliação

5.1 Resultados no Dataset de Treino

Métrica	Valor
Consistência	100%
Cobertura	≈ 100%
Compleitude	13.8%
Anomalias detectadas	31 (dia 7 vs. baseline dias 1-6)
Insights gerados (estratégia B)	4
Erro de parse estratégia A	Sim

5.2 Análise de Falhas

Compleitude baixa (13.8%). Como descrito na secção 2.5, este valor reflete uma limitação real do sistema de sensores simulado, não necessariamente um erro do algoritmo. Trajectórias que não têm evento de entrada numa porta Z_E são reconstruídas com o primeiro evento disponível, o que é a decisão correcta para maximizar cobertura, mas penaliza a completude.

Estratégia A falhou completamente. O **llama3.1:8b** sem exemplos não consegue produzir JSON estruturado de forma fiável. Em produção, ou se usaria sempre a estratégia B, ou se usaria um modelo maior (llama3.1:70b ou equivalente via API).

4 insights são poucos. O enunciado pede entre 8 e 12. Com um modelo maior ou temperatura ajustada, o número seria superior. Com o **llama3.1:8b** em hardware limitado, 4 insights foi o máximo obtido de forma estável.

5.3 Limitações

1. A cobertura de 100% é uma aproximação: o cálculo usa **journeys rows / entry events**, assumindo que cada linha de **journeys.csv** corresponde a um evento entry. Rastrear **event_id** originais no stitcher daria uma medida mais rigorosa.
2. O algoritmo de stitching não foi validado contra ground truth. Sem saber as trajectórias reais, não é possível calcular precision/recall da associação.
3. O **MAX_GAP_SECONDS = 120** foi escolhido empiricamente. Um valor diferente produziria um número diferente de visitantes únicos. A escolha afeta diretamente todas as métricas a jusante.

6. Conclusão

O que funcionou

O algoritmo de stitching atingiu 100% de consistência temporal, que é a invariante mais crítica do problema. A separação Python/LLM funcionou exatamente como planejado: todos os números no relatório final são rastreáveis ao **metrics.json** calculado em Python. A estratégia few-shot produziu insights com números concretos e recomendações acionáveis.

O que não funcionou

A estratégia zero-shot falhou completamente com o **llama3.1:8b**. A completude de 13.8% é baixa, refletindo a dificuldade de reconstruir trajetórias completas quando os sensores de porta não capturam todos os eventos. O número de insights gerados (4) ficou abaixo do pedido (8-12).

O que faria de diferente

Com mais tempo, implementaria: (1) rastreamento de event_id no stitcher para cobertura exata; (2) um mecanismo de votação demográfica para atributos (em vez de usar o primeiro valor disponível); (3) testes sistemáticos de MAX_GAP_SECONDS com diferentes valores para calibrar o número de visitantes contra benchmarks externos; (4) uso de um modelo LLM maior para melhorar a qualidade e quantidade dos insights.

Declaração de Utilização de Ferramentas de IA

No desenvolvimento deste projeto, recorri à assistência de ferramentas de Inteligência Artificial Generativa para otimizar o fluxo de trabalho e a qualidade dos entregáveis. A utilização destas ferramentas focou-se em três eixos principais:

1. **Desenvolvimento de Código:** Utilizei IA para auxiliar na estruturação dos algoritmos de *stitching* e nas métricas de análise em Python, garantindo a eficiência e o cumprimento das boas práticas de programação.
2. **Engenharia de Prompts:** As ferramentas de IA foram fundamentais para testar e refinar as estratégias *Zero-shot* e *Few-shot*, permitindo que o modelo qwen2.5:0.5b gerasse insights mais precisos e acionáveis.
3. **Documentação:** A IA auxiliou na revisão técnica do README.md e na organização da estrutura do relatório.